10/28/26

Groupe : Elio PIRONNEAU—SAURANUS, Lucas RIOS

Cahier des charges fonctionnel et technique

Projet fil rouge : plateforme de signalement cyber

1. Présentation du projet

1.1 Contexte

Le client est un prestataire fictif qui accompagne plusieurs organisations (entreprises, associations, établissements) sur leurs problèmes de cybersécurité. Aujourd'hui, les clients envoient leurs signalements par courriel ou dans des tableaux dispersés. Il est donc difficile de les retrouver, de les suivre et de voir où ils se situent.

1.2 Objectif

Créer une application web qui permet de :

- centraliser tous les signalements de problèmes cyber au même endroit ;
- suivre leur traitement (nouveau, en cours, clôturé) ;
- visualiser les signalements sur une carte interactive de la France métropolitaine.

1.3 Périmètre

L'application concerne la **France métropolitaine, Corse comprise**. La localisation d'un signalement correspond au **lieu concerné par le problème**. Toutes les données utilisées sont **fictives**.

Groupe : Elio PIRONNEAU—SAURANUS, Lucas RIOS

2. Les utilisateurs de l'application

Profil	Rôle
Administrateur	Travaille dans le back-office. Gère les clients, les types de problèmes, les signalements et les comptes utilisateurs. Voit tous les signalements.
Utilisateur client	Appartient à une organisation. Déclare des problèmes, consulte et suit les signalements de **son** organisation uniquement.
Visiteur	Non connecté. Peut seulement accéder à la page de connexion.

3. Besoins fonctionnels

3.1 Authentification et comptes

N°	Fonctionnalité	Priorité
F1	Se connecter avec un identifiant (courriel) et un mot de passe	Haute
F2	Se déconnecter	Haute
F3	Chaque utilisateur est redirigé selon son rôle (espace client ou back-office)	Haute
F4	Un compte suspendu ne peut plus se connecter	Haute
F5	Modifier son mot de passe	Moyenne

Groupe : Elio PIRONNEAU—SAURANUS, Lucas RIOS

3.2 Espace client

N°	Fonctionnalité	Priorité
C1	Déclarer un problème : description, type, localisation, date et heure des faits	Haute
C2	Voir la liste des signalements de son organisation	Haute
C3	Consulter le détail d'un signalement (statut, priorité, note de suivi ou de résolution)	Haute
C4	Voir les signalements de son organisation sur la carte	Haute
C5	Filtrer par type, état et période	Haute
C6	Modifier la localisation d'un de ses signalements tant qu'il est « nouveau »	Moyenne

Lors de la déclaration, l'application enregistre automatiquement **l'organisation** et **l'utilisateur** qui a fait la déclaration. Le statut initial est « Nouveau ».

3.3 Back-office administrateur

N°	Fonctionnalité	Priorité
A1	Créer, consulter, modifier et supprimer les **clients** (organisations)	Haute
A2	Créer, consulter, modifier et supprimer les **types de problèmes**	Haute
A3	Créer, consulter, modifier et supprimer les **signalements**	Haute
A4	Changer le statut d'un signalement (Nouveau, En cours, Clôturé)	Haute
A5	Définir la priorité d'un signalement (Basse, Moyenne, Haute, Critique)	Haute
A6	Ajouter une note de suivi ou de résolution visible par le client	Haute
A7	Créer les comptes utilisateurs d'un client et gérer leurs accès	Haute
A8	Suspendre ou réactiver un compte utilisateur	Haute
A9	Voir tous les signalements sur la carte et filtrer aussi par client	Haute
A10	Corriger les informations et la localisation d'un signalement	Haute

3.4 Carte interactive

N°	Fonctionnalité
M1	Afficher une carte Leaflet avec le fond OpenStreetMap, centrée sur la France métropolitaine (Corse incluse)
M2	Afficher un marqueur par signalement
M3	Au clic sur un marqueur, afficher les informations du signalement et un lien vers son détail
M4	Filtrer par type, état et période (et par client pour l'administrateur)
M5	La liste et la carte affichent **exactement les mêmes résultats** pour un même filtre
M6	Choisir ou modifier la localisation en cliquant sur la carte ou en déplaçant un marqueur (latitude et longitude)

Les marqueurs ont une couleur différente selon le statut (par exemple : rouge = nouveau, orange = en cours, vert = clôturé).

3.5 Exemple d'utilisation

1. Une organisation de Toulon reçoit des courriels suspects sur un de ses sites.
2. Son utilisateur se connecte, déclare le problème et place le marqueur sur Toulon.
3. Le signalement apparaît avec le statut « Nouveau ».
4. Un administrateur le prend en charge, fixe la priorité.
5. L'administrateur clôture le signalement avec une note de résolution.
6. Le client consulte son signalement et lit la note.

4. Besoins non fonctionnels

- **Fiabilité :** les données sont enregistrées dans une base de données et restent disponibles d'une utilisation à l'autre.
- **Sécurité :** voir la partie 7.
- **Facilité d'utilisation :** interface claire, lisible, avec des messages compréhensibles (erreurs, confirmations).
- **Adaptabilité (responsive) :** utilisable sur ordinateur et sur téléphone.
- **Performance :** les pages se chargent en quelques secondes avec les données de démonstration.
- **Maintenabilité :** code organisé et commenté, liste des types de problèmes modifiable sans toucher au code.

Groupe : Elio PIRONNEAU—SAURANUS, Lucas RIOS

5. Contraintes techniques

Élément	Choix
Langage serveur	Python
Framework web	Flask
Base de données (accès)	SQLAlchemy (ORM)
Base de données (moteur)	SQLite pour le développement et la démonstration
Interface	HTML, CSS et JavaScript
Carte	Leaflet avec fond OpenStreetMap
Modélisation	UML (diagrammes décrits en partie 8)
Versionnement	Git

6. Architecture de l'application

Nous utilisons une architecture simple en couches :

- **Modèles** (SQLAlchemy) : représentent les tables de la base de données.
- **Routes / vues** (Flask) : traitent les requêtes et renvoient les pages ou les données.
- **Templates** (HTML avec Jinja) : affichent les pages.
- **Fichiers statiques** : CSS, JavaScript, Leaflet.

Les routes sont séparées en trois groupes : authentification, espace client et back-office (préfixe `/admin`). Un petit service renvoie les signalements au format JSON pour alimenter la carte, avec les mêmes filtres que la liste.

Organisation prévue des fichiers :

- `app/models/` : classes de la base de données
- `app/routes/` : auth, client, admin, api
- `app/templates/` : pages HTML
- `app/static/` : css, js
- `README.md` : installation et utilisation

Groupe : Elio PIRONNEAU—SAURANUS, Lucas RIOS

7. Sécurité et confidentialité

- **Mots de passe** : jamais stockés en clair, ils sont hachés.
- **Sessions** : Flask-Login ; clé secrète stockée hors du code.
- **Contrôle des rôles** : toutes les pages `/admin` sont réservées aux administrateurs, y compris côté serveur.
- **Cloisonnement des clients** : chaque requête d'un utilisateur client est filtrée par son organisation. Cela vaut aussi pour la carte et le service JSON.
- **Compte suspendu** : refusé à la connexion et déconnecté s'il était déjà connecté.
- **Formulaires** : protection CSRF (Flask-WTF) et validation des champs.
- **Injection SQL** : évitée grâce à SQLAlchemy (pas de requêtes SQL écrites à la main).
- **XSS** : affichage des textes saisis protégé par l'échappement automatique de Jinja.
- **Localisation** : vérification que les coordonnées sont dans la France métropolitaine.
- **Données** : uniquement fictives ; aucune donnée personnelle réelle.

8. Modélisation prévue (UML)

8.1 Diagramme de cas d'utilisation

- Acteur **Utilisateur client** : se connecter, déclarer un problème, consulter ses signalements, filtrer, consulter la carte, suivre le traitement.
- Acteur **Administrateur** : se connecter, gérer les clients, gérer les types, gérer les signalements, changer statut et priorité, ajouter une note, gérer et suspendre les comptes, consulter la carte globale.

8.2 Diagramme de classes (modèle de données)

Client

- id, nom, type, ville, date de création

Utilisateur

- id, nom, prénom, courriel, mot de passe haché, rôle, actif, id du client

TypeProbleme

- id, libellé, description, actif

Groupe : Elio PIRONNEAU—SAURANUS, Lucas RIOS

Signalement

- id, titre, description, date et heure des faits, date de création, latitude, longitude, adresse, statut, priorité, note de suivi, id du client, id de l'utilisateur déclarant, id du type

Relations :

- un Client possède 1 à plusieurs Utilisateurs ;
- un Client possède 0 à plusieurs Signalements ;
- un Utilisateur déclare 0 à plusieurs Signalements ;
- un TypeProbleme est associé à 0 à plusieurs Signalements.

8.3 Diagramme de séquence

Scénario « Déclarer un signalement » : l'utilisateur remplit le formulaire, Flask vérifie la session et les données, SQLAlchemy enregistre dans la base, l'utilisateur est redirigé vers le détail.

8.4 Diagramme d'états d'un signalement

Nouveau → En cours → Clôturé

9. Interface

Pages principales :

- Connexion
- Espace client : tableau de bord, liste des signalements avec filtres, carte, formulaire de déclaration, détail d'un signalement
- Back-office : tableau de bord, gestion des clients, des types, des utilisateurs et des signalements, carte globale

Principes :

- menu simple et identique sur toutes les pages ;
- messages clairs (« Votre signalement a bien été enregistré », « Ce champ est obligatoire ») ;
- confirmation avant toute suppression ;
- couleurs de statut accompagnées d'un texte, pas seulement de la couleur.